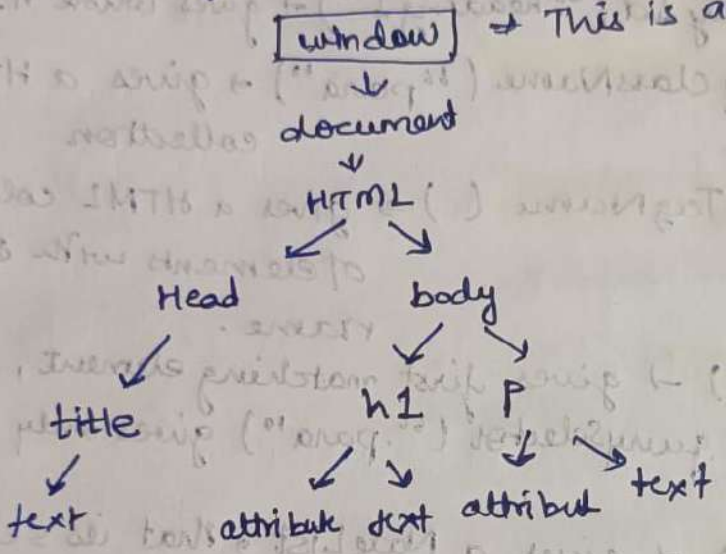
DOM

DOM ⇒ Document Object Model : A tree representation of all HTML elements.

⇒ Dom gives us interface, which we can use to access and manipulate HTML elements.

⇒ Window is the global object of browser.



ElementNode ⇒ The nodes which are elements of HTML

TextNode ⇒ having text / value

CommentNode ⇒ Targets comments

AttributeNodes ⇒ having id and other all

DocumentNode ⇒

⇒ getElementById ⇒ select an element which has id = " ".

All elementNode has "nodeType" = 1
All attributeNode has "nodeType" = 2
All TextNode has "nodeType" = 3
All CommentNode has "nodeType" = 9
All DocumentNode has "nodeType" = 8

Interfaces are used to access and manipulate DOM.

DOM methods
- → CRUD elements
- → class Manipulation
- → Styling
- → Selecting
- → Reading & changing content
- → attributes

⇒ Selecting Elements Using DOM

1. getElementById("heading1") → gives whole h1 tag (single element)
2. getElementsByClassName("para") ⇒ gives a HTML collection
3. getElementsByTagName() ⇒ gives a HTML collection of elements with that tag name.
4. querySelector() → gives first matching element,
   eg document.querySelector(".para") gives only 1 element
5. querySelectorAll() → gives a NodeList, that is selects all elements that match a CSS selector

Difference b/w ~~query selector~~ NodeList and HTML collection.

→ NodeList → collection of DOM nodes and is mostly static
- → Returned when .querySelectorAll
- → .forEach is available in NodeList

→ HTML collection ⇒ collection of HTML element node only
- → Has only ElementNode
- → Returned by getElementsByTagName
- → Returned by getElementsByClassName

⇒ HTML collection is dynamic / live

Live here means if the DOM changes, the collection updates automatically

⇒ Assignment 1 : Diff btw NodeList and HTML collection

⇒ Reading and changing content using DOM

1. document.getElementById("heading1").innerText gives the text inside the h1 tag.
2. document.getElementById("heading1").textContent
3. document.getElementById("heading1").innerHTML

Text Content	Inner HTML	Inner Text
⇒ returns all the text including hidden text eg display: none	⇒ returns visible content along with HTML tags	⇒ only brings visible text and no tags
⇒ does not considers CSS styling	⇒ HTML + text	⇒ considers CSS styling also

⇒ Attributes

⇒ . getAttribute ⇒ reads the value of an attribute, also returns the value of attribute

⇒ . setAttribute ⇒ sets or update an attribute. takes

⇒ . setAttribute ("attributeName", "value")

⇒ does not return anything

⇒ .removeAttribute() ⇒ used to delete/remove an attribute

⇒ Styling

.style is used to add/change CSS styles of an HTML element

eg. const el = document.getElementById("text");
el.style.color = "red"

⇒ This code above changes the color of the text with id "text" to pink.

eg. el.style.fontSize = "30px"; etc

Assignment: Research DOM TokenList

⇒ Class Manipulation

JS can add, remove, toggle or check classes of HTML element using classList.

eg. .add() ⇒ to add a class
.remove() ⇒ to remove a class
.toggle() ⇒ adds if its missing, remove if already present. Used when we want to use remove and add both.
.contains() ⇒ checks if a class exists or not.

1. DARK MODE TOGGLE

⇒ Make a HTML file, CSS and JS, and link it.

⇒ Create a container tag, a button has a id
Here container has a class

⇒ Target button with its id, and store in a variable eg btn

⇒ btn.addEventListener("click", () ⇒ {
document.body.classList.toggle("dark")

3)

⇒ Add .dark with bg : #121212 and color : white

⇒ Add .dark button {
background : white;
color : black;

}

Assigment : If we are in dark mode "Text : Toggle To Light Mode"
Make the text dynamic

2. Dynamic List Creator

⇒ Make a div with class "container", a h1, and a div with text and button (~~for add and remove~~)

⇒ Also make a unordered list tag.

⇒ Now select input box and store in a variable
Now select button

⇒ Add click, event listener, also add validation to check value should not be empty.

⇒ Create a new element using createElement.

⇒ Also add value of the text using input.value.

Assignment : Add Edit button, only text is clicked twice, text Input becomes editable

- Now we need to attach/add to ul, by using
  list.appendChild(li)

- Now we need to clear text, so we do:
  input.value = " "

- Now we need to remove a text, so we ~~select~~ create the Delete button,
  ⇒ Now we need to add ~~text~~ textContent = "Delete";
  - Also add a classList.add("delete");
  - Now we need to append into ul:
    ~~list~~ li.appendChild(delBtn);
  - Add an event listner to delete the task added.

```
delBtn.addEventListener("click", () => {
        li.remove();
})
```

3. Color Palette Generator

We have colors in hex, or rgb format.

⇒ Select options using their ids, also select the button

- Add a eventlistner to the selected button,
- Convert rgb to hex
- Create a fun randomRGB on basis of tone